

1. Keywords: this and super

this **Keyword:** Refers to the **current object** of the class. It is used to:

- Distinguish between instance variables and parameters with the same name (this.name = name)
- Call one constructor from another constructor in the same class (this())
- Return the current instance from a method (return this)

super **Keyword:** Refers to the **parent class object**. It is used to:

- Access parent class variables hidden by child class variables (super.parentVar)
- Call parent class methods that are overridden (super.display())
- Invoke parent class constructor from child constructor (super() - must be first statement)

java

```
class Parent {
    int value = 10;
    Parent() { System.out.println("Parent Constructor"); }
}

class Child extends Parent {
    int value = 20;
    void show() {
        System.out.println(this.value); // 20 (current class)
        System.out.println(super.value); // 10 (parent class)
    }
}
```

2. Garbage Collection and finalize() Method

Garbage Collection (GC): Automatic memory management process in Java that **removes unreferenced objects** from heap memory. When an object has no active references pointing to it,

GC identifies it as "garbage" and destroys it, freeing memory. Programmers cannot force GC, but can request it using `System.gc()`.

`finalize()` **Method:** A **protected method** in `Object` class that is **called by GC just before destroying an object**. It gives an object a last chance to perform cleanup operations like closing files, database connections, or releasing system resources. After `finalize()` executes, the object is permanently removed.

```
java
class Resource {
    protected void finalize() {
        System.out.println("Object destroyed, cleanup done");
    }
}
```

3. Purpose of Package and How to Import

Purpose of Package: Packages are **folders/directories** that organize related classes and interfaces. Benefits include:

- **Prevents naming conflicts** (e.g., `java.util.Date` vs `java.sql.Date`)
- **Provides access protection** (protected and default access levels work within package)
- **Makes code organization and maintenance easier**
- **Enables code reusability**

How to Import a Package:

Import Type	Syntax	Example
Import specific class	<code>import packageName.ClassName;</code>	<code>import java.util.Scanner;</code>

Import Type	Syntax	Example
Import entire package	<code>import packageName.*;</code>	<code>import java.util.*;</code>
Static import	<code>import static packageName.ClassName.staticMember;</code>	<code>import static java.lang.Math.PI;</code>
Fully qualified name (no import needed)	Use complete path each time	<code>java.util.Scanner sc = new java.util.Scanner(System.in);</code>

Default packages automatically imported: `java.lang.*`

Example:

```
java
```

```
// At the top of your Java file
```

```
package myproject.students;    // Declaring package
```

```
import java.util.ArrayList;    // Importing specific class
```

```
import java.io.*;              // Importing all classes from java.io
```

```
public class StudentDemo {
```

```
    ArrayList<String> names = new ArrayList<>();
```

```
}
```